Submission Worksheet

Submission Data

Course: IT202-008-S2026
Assignment: IT202 Module 4 SQL Challenge
Student: Deep S. (dns33)
Status: Submitted | **Worksheet Progress:** 100+%
Potential Grade: 11.00/10.00 (110.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 3/23/2026 7:21:30 PM
Updated: 3/26/2026 6:26:03 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-module-4-sql-challenge/grading/dns33
View Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-module-4-sql-challenge/view/dns33

Instructions

- Overview Link: https://youtu.be/oW7AK39mgmo

1. Ensure you read all instructions and objectives before starting.
2. Create a new branch from `qa` called `M4-Homework`
    1. `git checkout qa` (ensure proper starting branch)
    2. `git pull origin qa` (ensure history is up to date)
    3. `git checkout -b M4-Homework` (create and switch to branch)
3. Copy the template code from here: GitHub Repository - M4 Homework
    - Put all into an `M4` folder inside your `public_html`
    - Expected files
        - [ ] sql/000_create_table_todos.sql
        - [ ] sql/init_db.php
        - [ ] todos/create.php
        - [ ] todos/pending.php
        - [ ] todos/completed.php
        - [ ] nav.php
        - [ ] two index files (M4 and todos)
    - Immediately record to history
        - [ ] `git add public_html`
        - [ ] `git commit -m "adding M4 HW baseline files"`
        - [ ] `git push origin M4-Homework`
        - [ ] Create a Pull Request from `M4-Homework` to `qa` and keep it open
4. Fill out the below worksheet
    - Each Problem requires the following as you work
        - [ ] Ensure there's a comment with your UCID, date, and brief summary of how the problem was solved
        - [ ] Update ucid in nav.php where noted
        - [ ] Code solution (add/commit periodically as needed) (refer to comments in the create, pending,

completed files)

5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin M4-Homework`
    4. On Github merge the pull request from `M4-Homework` to `qa`
    5. On Github create a pull request from `qa` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the render.com prod links work)
7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout qa`
    2. `git pull origin qa`

Section #1: (1 pt.) Prep

Progress: 100%

Details:
Once the files are copied in, run your local PHP server and visit `/M4/sql/init_db.php` in the browser to trigger the table generation.

Task #1 (0.50 pts.) - Show the output of the init_db.php tool

Progress: 100%

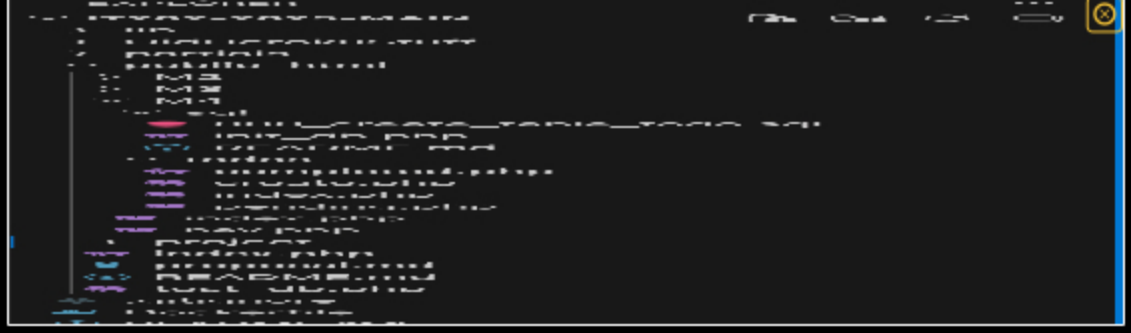
Details:
The output should note either successfully executed or skipped (if you ran this already)

Screenshot

Saved: 3/23/2026 7:22:12 PM

Task #2 (0.50 pts.) - Show the project view of your editor to show all files are present

Progress: 100%



Vs code files

Saved: 3/23/2026 7:24:07 PM

Section #2: (3 pts.) Create Page

Progress: 100%

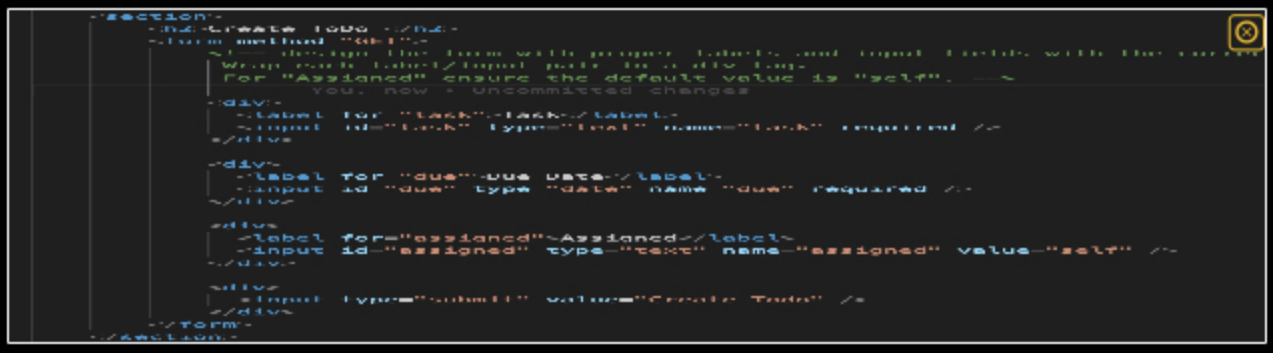
Task #1 (0.75 pts.) - HTML Form Challenge

Progress: 100%

Part 1:

Progress: 100%

Details:
Show the code snippet related to the completed HTML form



code snippet

Saved: 3/25/2026 5:36:57 PM

Part 2:

Progress: 100%

Details:
Items

- Note the field names and why the specific types were chosen
- Also note how you made "self" the default for the assigned input

Your Response:

Field names and why types were chosen -task uses type="text" because it's a short description that the user types in. -due uses type="date" so the browser provides a date picker and the value submits in a valid date format. -assigned uses type="text" because it stores who the task is assigned to (a name/label).

How "self" is the default -I set the assigned input's default using value="self" so it is pre-filled automatically but the user can still edit it if needed.

Saved: 3/25/2026 5:36:57 PM

Task #2 (0.75 pts.) - Form Submission PHP Validations

Progress: 100%

Part 1:

Progress: 100%

Details:

- Validate the incoming data for correct format based on the SQL table definition.
- When not valid, provide a user-friendly message of what specifically was wrong and set `$is_valid` to false
- Assigned should check for "self" if a valid format/value isn't provided; it should default to "self" if all checks for this property fail
- Include screenshot(s) from Render.com QA demonstrating each validation (ensure URL is visible)

Invalid Date

Task Empty

Empty Assigned

Saved: 3/26/2026 5:36:25 PM

Part 2:

Progress: 100%

Details:
Briefly explain your validation logic.

Your Response:

I first trim the inputs to remove extra spaces. For task, I check it isn't empty and I also limit the length so it doesn't exceed the column size. For due, I verify it matches the YYYY-MM-DD format using DateTime::createFromFormat, and I reject it if the parsed date doesn't match the original string. For assigned, if it's blank (or looks like "null/undefined") I default it to "self"; otherwise I only accept it if it matches a safe pattern (letters/numbers/spaces/underscore/dash) and default to "self" if it fails. If any errors exist, I set $is_valid = false and print a user-friendly list of messages.

Saved: 3/26/2026 5:36:25 PM

Task #3 (0.75 pts.) - Show the code snippet related to the INSERT query

Progress: 100%

Part 1:

Progress: 100%

Details:
Design a query to insert the incoming data to the proper columns. Ensure valid and proper PDO named placeholders are used. https://phpdelusions.net/pdo

```
$query = "INSERT INTO M4_Todos (task, due, assigned) VALUES (:task, :due, :assigned)";
$params = [
    ":task" => $task
```

```
    ":due" => $due,
    ":assigned" => $assigned
];
```

@var string $assigned

You, 3 minutes ago • Uncommitted changes

SQL snippet

Saved: 3/25/2026 5:07:00 PM

Part 2:

Progress: 100%

Details:
Briefly describe `how` the query works in plain words.

Your Response:

I built an INSERT query using named placeholders (:task, :due, :assigned). Then I mapped each placeholder to the PHP variables in the $params array. PDO prepares the query first, then executes it with the mapped values, which inserts a new row into the todos table. After it runs, I show the new id using lastInsertId().

Saved: 3/25/2026 5:07:00 PM

Task #4 (+ 1.01 pts.) - Extra Credit: Handle Unique Constraint Errors

Progress: 100%

Part 1:

Progress: 100%

Details:
Show the code related to handling the unique constraint errors and the user-friendly message

```
        } catch (PDOException $e) {
            // Extra credit: unique constraint (task + due)
            // MySQL duplicate entry error code is 1062
            if (isset($e->errorInfo[1]) && $e->errorInfo[1] == 1062) {
                echo "<div style='color:red; font-weight:bold;'>
                        That task already exists for that due date. Try a different task or date.
                      </div>";
            } else {
                echo "There was an error inserting the record; check the logs (terminal)";
            }
            error_log("Insert Error: " . var_export($e, true));
        }
    } else {
        error_log("Creation input wasn't valid");
    }
```

You, yesterday • Added M4 homework files

Code Snippet

Saved: 3/25/2026 5:50:14 PM

Part 2:

Progress: 100%

Details:
Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

In the catch block, I check the PDO error info for MySQL error 1062, which means a duplicate entry violated the unique constraint. If it matches, I print a user-friendly message explaining the task and due date already exist. Otherwise, I fall back to the generic error message and still log the full exception for debugging.

Saved: 3/25/2026 5:50:14 PM

Task #5 (0.75 pts.) - Create Page URLs

Progress: 100%

Details:

- Direct link to the file in the homework-related branch from Github (should end in `.php` , , zero credit if it does not)
- Direct link to the file on Render.com Prod (Grab the qa url you're testing, paste it, change "-qa" to "-prod"), zero credit if not prod

URL #1
https://github.com/shahdp2/IT202-SP2026/blob/M4-Homework/public_html/M4/todos/create.php

URL
https://github.com/shahdp2/

URL #2
https://dns33-it202-008-prod.onrender.com/M4/todos/create.php

URL
https://dns33-it202-008-prod.

Saved: 3/26/2026 5:41:06 PM

Section #3: (3 pts.) Pending Page

Progress: 100%

Task #1 (1 pt.) - Fetching ToDos

Progress: 100%

Part 1:

Progress: 100%

Details:

- Refer to the HTML table in the given code and build a query that'll select the columns in the same order as the table from the Todo table
- Cross-reference the HTML table columns with what'd most plausibly match the SQL table, aside from the notes below
- For the Status part, you'll need to calculate the "days_offset" from the due date, ensure the virtual column matches "days_offset"
- For Actions, this isn't part of the query, and there's nothing special to select for it
- Filter the results where the todo item is NOT completed and order the results by those due soonest
- No limit is required
- Include screenshot(s) from Render.com QA demonstrating 5 different records (ensure URL is visible)

Pending qa page

Saved: 3/26/2026 5:52:26 PM

Part 2:

Progress: 100%

Details:
Briefly describe `how` the query works in plain words.

Your Response:

The query selects each pending todo's id, task, due date, and assigned value. It also calculates days_offset using DATEDIFF(due, CURRENT_DATE) to show how many days away the due date is. It filters only records where is_complete = 0 and sorts the results by the due date so the soonest items appear first.

Saved: 3/26/2026 5:52:26 PM

Task #2 (1 pt.) - Completing ToDos

Progress: 100%

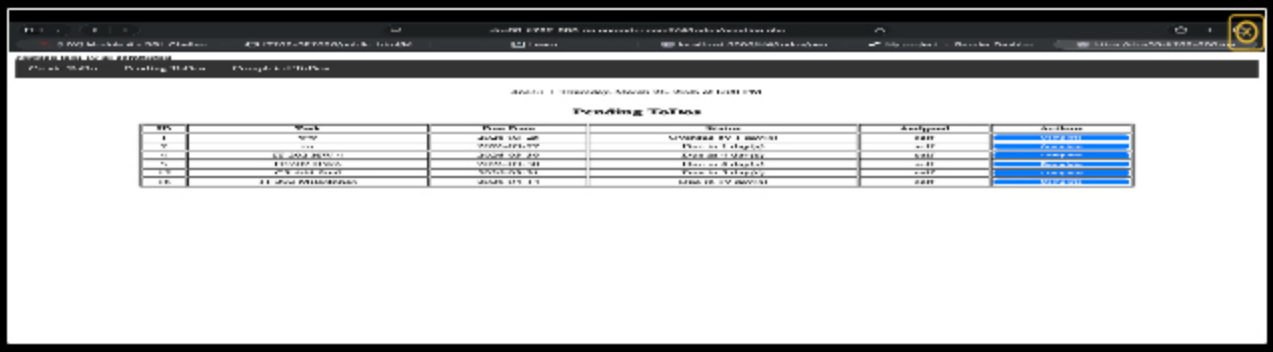
Part 1:

Progress: 100%

Details:

- Create a query that'll update the respective ToDo, marking it complete and setting the date for the completed date field as today
- Ensure the "id" is utilized using proper PDO named placeholders so that only the one item is updated
- Add an extra clause to update only if the complete field of the record is not set

Completed Page



Pending page

Saved: 3/26/2026 6:02:19 PM

Part 2:

Progress: 100%

Details:
Briefly describe `how` the query works in plain words.

Your Response:

This query updates a single todo row by matching its id using a named placeholder :id. It sets is_complete to 1 and stores today's date in the completed date column. The extra condition AND is_complete = 0 ensures it only updates the record if it hasn't already been marked complete.

Saved: 3/26/2026 6:02:19 PM

Task #3 (1 pt.) - Pending Page URLs

Progress: 100%

Details:

- Direct link to the file in the homework-related branch from GitHub (should end in `.php`, , zero credit if it does not)
- Direct link to the file on Render.com Prod (Grab the qa url you're testing, paste it, change "-qa" to "-prod"), zero credit if not prod

URL #1
https://github.com/shahdp2/IT202-SP2026/blob/M4-Homework/public_html/M4/todos/pending.php

URL
https://github.com/shahdp2/

URL #2
https://dns33-it202-008-prod.onrender.com/M4/todos/pending.php

URL
https://dns33-it202-008-prod.

Saved: 3/26/2026 5:53:57 PM

Section #4: (2 pts.) Completed Page

Progress: 100%

Task #1 (1 pt.) - Fetching ToDos

Progress: 100%

Part 1:

Progress: 100%

Details:

- Refer to the HTML table below and build a query that'll select the columns in the same order as the table from the Todo table
- Cross-reference the HTML table columns with what'd most plausibly match the SQL table, aside from the notes below
- For the completed date, you'll need to extract the date portion from the completed column
- For the Status part, you'll need to calculate the "days_offset" from the completed date, ensure the virtual column matches "days_offset"
- Filter the results where the todo item is completed and order the results by most recently completed and most recently due
- No limit is required
- Include screenshot(s) from Render.com QA demonstrating 5 different records (ensure URL is visible)

completed tasks

Saved: 3/26/2026 6:09:39 PM

Part 2:

Progress: 100%

Details:
Briefly describe `how` the query works in plain words.

Your Response:

My query selects completed todo items from the M4_Todos table. It returns the columns in the same order as the HTML table. I convert the completed timestamp into a date-only value using DATE(completed), and I calculate days_offset as the number of days since the todo was completed using DATEDIFF(CURRENT_DATE, DATE(completed)). Finally, it sorts results so the most recently completed items appear first, and if there's a tie it shows the ones with the most recent due dates first.

Saved: 3/26/2026 6:09:39 PM

Task #2 (1 pt.) - Completed Page URLs

Progress: 100%

Details:

- Direct link to the file in the homework-related branch from GitHub (should end in `.php` , , zero credit if it does not)
- Direct link to the file on Render.com Prod (Grab the qa url you're testing, paste it, change "-qa" to "-prod"), zero credit if not prod

URL #1
https://github.com/shahdp2/IT202-SP2026/blob/M4-Homework/public_html/M4/todos/completed.php

URL
https://github.com/shahdp2/

URL #2
https://dns33-it202-008-prod.onrender.com/M4/todos/completed.php

URL
https://dns33-it202-008-prod.

Saved: 3/26/2026 6:10:18 PM

Section #5: (1 pt.) Misc

Progress: 100%

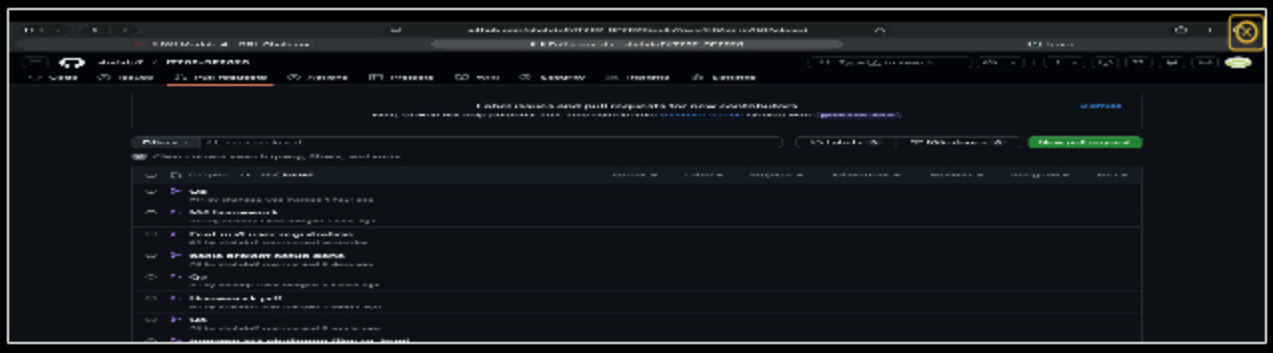
Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



commit history screenshot

Saved: 3/26/2026 6:23:21 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/shahdp2/IT202-SP2026/pull/11

URL
https://github.com/shahdp2/IT20

Saved: 3/26/2026 6:23:21 PM

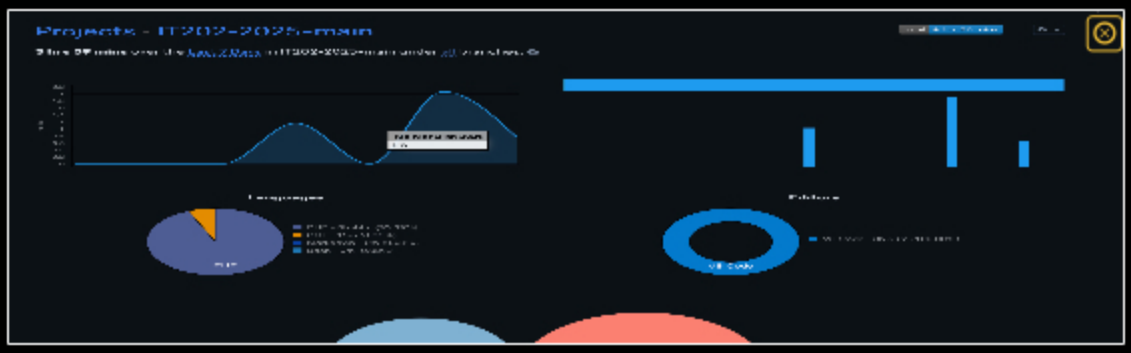
Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard

- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



wakatime 1

wakatime 2

Saved: 3/26/2026 6:24:47 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to connect PHP to a MySQL database using PDO and how to run SQL setup files to create tables. I also learned how to build SQL queries for inserting, selecting, filtering, ordering, and updating records. On top of that, I got better at validating form input in PHP so bad data doesn't get inserted into the database.

Saved: 3/26/2026 6:25:45 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was setting up the baseline files and running init_db.php to create the table. Creating the basic HTML form fields for task, due date, and assigned was also straightforward. Once the table structure was clear, writing the simple INSERT query with placeholders was not too bad.

Saved: 3/26/2026 6:25:52 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was getting the SQL queries to match the table output exactly, especially calculating the days_offset and formatting the status messages correctly. Handling edge cases like invalid dates, empty values, and the unique constraint error took the most time. Debugging issues between local vs Render and making sure everything worked the same in QA was also a little tricky.

Saved: 3/26/2026 6:26:03 PM